

# Hermes Kanban

## Multi-Agent Profile Collaboration for Hermes Agent

*A durable, profile-aware work-queue architecture for coordinating  
heterogeneous agent workloads across research, operations, and engineering*

Hermes & Teknium  
Nous Research

April 25, 2026 | Design Spec, Revision 01

### Abstract

This document specifies **Hermes Kanban**, a multi-agent collaboration layer for the Hermes Agent framework. It defines a durable, SQLite-backed task board shared across named agent *profiles*, allowing heterogeneous agents—researchers, editors, engineers, persistent digital-twin assistants—to collaborate asynchronously on long-lived work without the lifecycle fragility of in-process subagent swarms.

We compare the proposed architecture against three contemporary systems: Cline Kanban’s worktree-per-task board, Paperclip’s persistent-agent company orchestrator, and NanoClaw’s failed in-process agent-team SDK. We further contextualize the design against Google’s April 2026 Gemini Enterprise Agent Platform, which splits the agentic stack between a governance-oriented control plane and a velocity-oriented execution plane. Hermes Kanban deliberately stays at the execution plane, with governance expressible as user-space profiles and plugins rather than kernel features.

The specification covers: a minimal SQLite schema (tasks, links, comments, events), a dumb cron-driven dispatcher with atomic claims, eight canonical collaboration patterns, four user stories spanning research / scheduled operations / persistent digital twins / coding, a first-class orchestrator profile pattern (addressing the common “my orchestrator does the work itself” failure mode), an optional single-column multi-tenant extension that lets one specialist fleet serve many business contexts, and an implementation plan organized as eight independently shippable pull requests. The kernel requires no changes to `run_agent.py`, no new core tools, and no tool-schema expansion on any API call.

---

*Status: DESIGN ONLY. No implementation accompanies this document.  
Target audience: Hermes maintainers and contributors; design-review participants.*

# Contents

|          |                                                                |           |
|----------|----------------------------------------------------------------|-----------|
| <b>1</b> | <b>Motivation</b>                                              | <b>4</b>  |
| 1.1      | The multi-agent gap in Hermes today                            | 4         |
| 1.2      | What others have built, and what broke                         | 4         |
| 1.3      | Google Gemini Enterprise: control vs. execution                | 5         |
| 1.4      | Design thesis                                                  | 5         |
| <b>2</b> | <b>Related Systems: Comparative Analysis</b>                   | <b>7</b>  |
| <b>3</b> | <b>Architecture</b>                                            | <b>8</b>  |
| 3.1      | Overview                                                       | 8         |
| 3.2      | Three-plane separation                                         | 8         |
| 3.3      | The critical invariant: no in-process subagent swarms          | 8         |
| <b>4</b> | <b>Data Model</b>                                              | <b>9</b>  |
| 4.1      | SQLite schema                                                  | 9         |
| 4.2      | Status semantics                                               | 10        |
| 4.3      | Workspace kinds                                                | 10        |
| <b>5</b> | <b>Collaboration Patterns</b>                                  | <b>11</b> |
| 5.1      | P1 — Fan-out                                                   | 11        |
| 5.2      | P2 — Pipeline                                                  | 11        |
| 5.3      | P3 — Voting / Quorum (fan-in)                                  | 11        |
| 5.4      | P4 — Long-running journal                                      | 11        |
| 5.5      | P5 — Human-in-the-loop triage                                  | 11        |
| 5.6      | P6 — @mention delegation                                       | 11        |
| 5.7      | P7 — Thread-scoped workspace                                   | 11        |
| 5.8      | P8 — Fleet farming                                             | 12        |
| <b>6</b> | <b>The Orchestrator Profile</b>                                | <b>13</b> |
| 6.1      | Three properties of a well-behaved orchestrator                | 13        |
| 6.2      | Shipping implication: installable profile templates            | 13        |
| 6.3      | Why this is not kernel work                                    | 14        |
| <b>7</b> | <b>Multi-Tenant Context</b>                                    | <b>15</b> |
| 7.1      | The tenant primitive                                           | 15        |
| 7.2      | What “tenant” actually means                                   | 15        |
| 7.3      | What stays shared across tenants                               | 15        |
| 7.4      | What is deliberately <i>not</i> here                           | 15        |
| 7.5      | The “small company serving multiple clients” ask, solved       | 16        |
| <b>8</b> | <b>Worked Example: 50-Account Social Media Fleet</b>           | <b>17</b> |
| 8.1      | Setup                                                          | 17        |
| 8.2      | Per-tick task generation                                       | 17        |
| 8.3      | The dispatcher does what it does                               | 17        |
| 8.4      | Why this is clean                                              | 18        |
| <b>9</b> | <b>User Stories</b>                                            | <b>19</b> |
| 9.1      | Story 1: Research triage and synthesis (non-coding)            | 19        |
| 9.2      | Story 2: Scheduled recurring workflow (non-coding)             | 20        |
| 9.3      | Story 3: Digital-twin / persistent assistant role (non-coding) | 20        |
| 9.4      | Story 4: Coding pipeline (code-shaped)                         | 20        |

|                                                 |           |
|-------------------------------------------------|-----------|
| 9.5 What all four stories share . . . . .       | 20        |
| <b>10 Kanban vs. <code>delegate_task</code></b> | <b>22</b> |
| <b>11 Assignment Semantics</b>                  | <b>23</b> |
| <b>12 Dispatcher Design</b>                     | <b>24</b> |
| 12.1 Concurrency correctness . . . . .          | 24        |
| <b>13 CLI and Gateway Surface</b>               | <b>25</b> |
| 13.1 Command surface . . . . .                  | 26        |
| 13.2 Gateway integration . . . . .              | 27        |
| <b>14 Scope Boundaries</b>                      | <b>28</b> |
| <b>15 Implementation Plan</b>                   | <b>29</b> |
| 15.1 Files touched . . . . .                    | 29        |
| <b>16 Risks, Tradeoffs, Open Questions</b>      | <b>31</b> |
| 16.1 Risks . . . . .                            | 31        |
| 16.2 Tradeoffs . . . . .                        | 31        |
| 16.3 Open questions . . . . .                   | 31        |
| <b>17 Conclusion</b>                            | <b>32</b> |